LAB S09. Mensajería con RabbitMQ en Mexi-Banco

Javier Velasco Pacheco

Grupo 2

Sistemas Distribuidos 2027-1

PARTE 1. Ver el problema

Se ingresó al panel de administración de RabbitMQ desde el navegador, mediante la dirección http://localhost:15672, utilizando las credenciales proporcionadas. En las pestañas Exchanges y Queues and Streams se reviso la configuración inicial del broker para identificar los intercambios y las colas disponibles antes de realizar las pruebas.

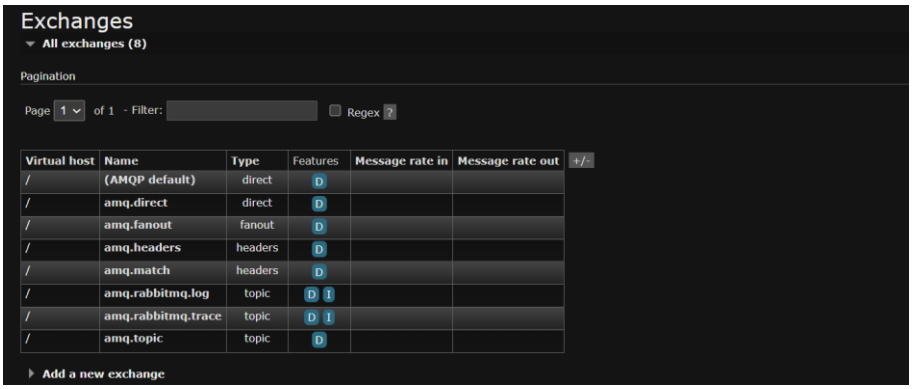


Figura. Panel de Exchanges de RabbitMQ antes de realizar las pruebas.

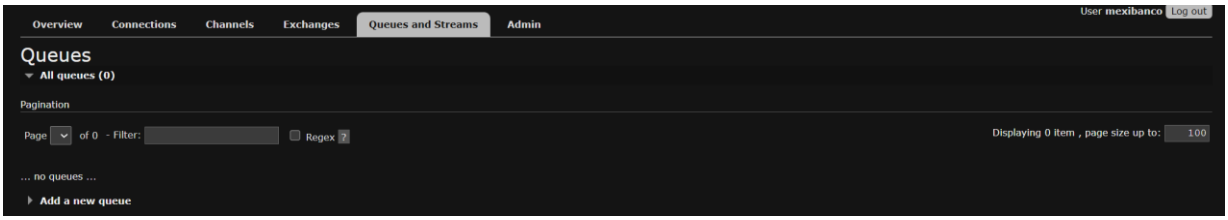


Figura. Panel de Queues de RabbitMQ antes de realizar las pruebas.

Para realizar las pruebas se utilizaron dos cuentas creadas mediante el gateway de Mexi-Banco. Como las CLABEs originales ya estaban registradas en la base de datos, se utilizaron CLABEs diferentes para evitar conflictos.

La cuenta de Ana se creó con la CLABE 002180000000000003 y un saldo inicial de $1,000.00. La cuenta de Beto se creó con la CLABE 002180000000000004 y un saldo inicial de $500.00. Posteriormente, la consulta de ambas cuentas confirmó los siguientes datos:

Cuenta	CLABE	Saldo inicial
Ana	002180000000000003	$1,000.00
Beto	002180000000000004	$500.00

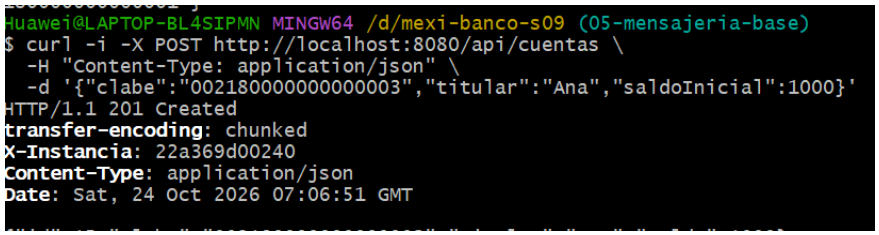


Figura. Cuenta Ana

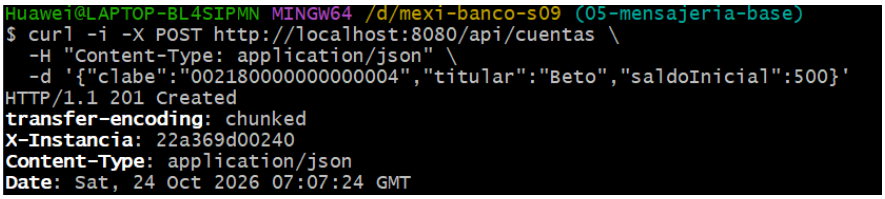


Figura. Cuenta Beto

Primero se detuvo el servicio de notificaciones mediante el comando docker compose stop notificacion. Después, se intentó realizar una transferencia de $50.00 desde la cuenta de Ana hacia la cuenta de Beto, utilizando el endpoint POST /api/transferencias.

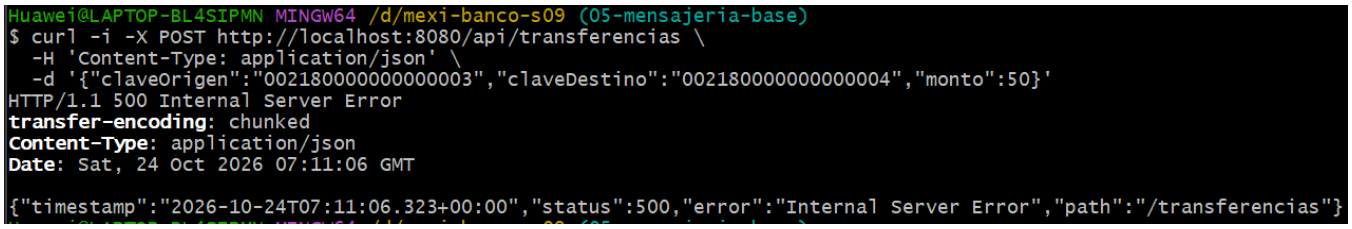


Figura. Error 500

El servidor respondió con el código 500 Internal Server Error, por lo que la operación no pudo confirmarse como exitosa desde la respuesta del cliente. Es necesario revisar los registros del servicio de transferencias y consultar los saldos para confirmar qué ocurrió internamente.

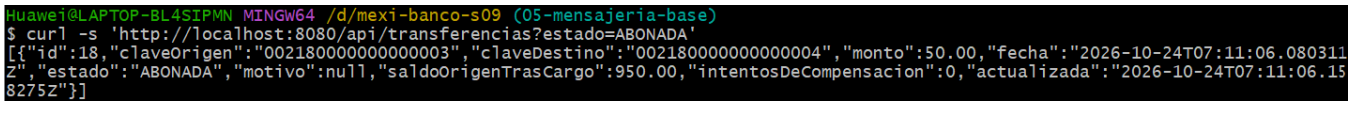


Figura. Registros de transferencias.

El cliente recibe un error porque la saga estaba esperando que el servicio de notificaciones respondiera correctamente y, al estar apagado, falló aunque el cargo ya se habían realizado, por eso el cambios en los saldos.

```
Huawei@LAPTOP-BL4SIPMN MINGW64 /d/mexi-banco-s09 (05-mensajeria-base)
$ curl -s http://localhost:8080/api/cuentas/002180000000000003
{"id":15,"clabe":"002180000000000003","titular":"Ana","saldo":950.00}
Huawei@LAPTOP-BL4SIPMN MINGW64 /d/mexi-banco-s09 (05-mensajeria-base)
$ curl -s http://localhost:8080/api/cuentas/002180000000000004
{"id":16,"clabe":"002180000000000004","titular":"Beto","saldo":550.00}
Huawei@LAPTOP-BL4SIPMN MINGW64 /d/mexi-banco-s09 (05-mensajeria-base)
$
```

Figura. Cambio de Saldos

PARTE 2. Publicar el evento

En application.yml se activaron las confirmaciones de RabbitMQ y la devolución de mensajes que no pueden enrutarse:

```
spring:
  rabbitmq:
    publisher-confirm-type: correlated
    publisher-returns: true
    template:
      mandatory: true
```

En PublicadorDeEventos.java, el código envía el evento a RabbitMQ y espera una respuesta para saber si el mensaje fue aceptado. Si ocurre un error, se acaba el tiempo de espera o ninguna cola puede recibir el mensaje, se registra el problema.

En TransferenciaService.java, el método avisar envía el evento después de guardar la transferencia. Si el envío falla, se registra el error, pero la transferencia sigue completada.

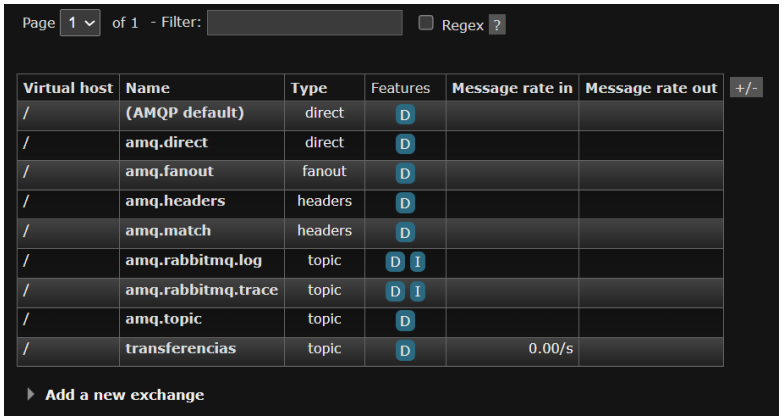


Figura. Configuración del exchange transferencias con tipo topic.

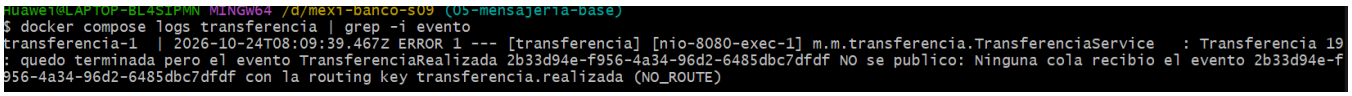


Figura. log con el error NO_ROUTE

La confirmación ack indica que RabbitMQ aceptó el mensaje, pero no asegura que alguien lo reciba o procese. En nuestra prueba, NO_ROUTE apareció porque no había ninguna cola conectada al Exchange.

PARTE 3. Consumir el evento

Se inició el servicio notificacion para declarar la cola.. Después, se detuvo el servicio y se realizaron tres transferencias de Ana a Beto por $11, $12 y $13. Las solicitudes debían responder con HTTP 201.



Figura. Las tres transferencias con respuestas HTTP 201

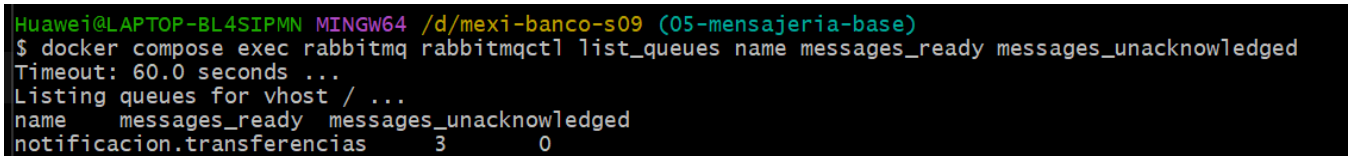


Figura. La cola con tres mensajes pendientes y cero mensajes sin confirmar.

Se verificó que RabbitMQ almacenara los tres mensajes pendientes y, al reiniciar notificacion, estos se procesaran y aparecieran en las bandejas de Ana y Beto.

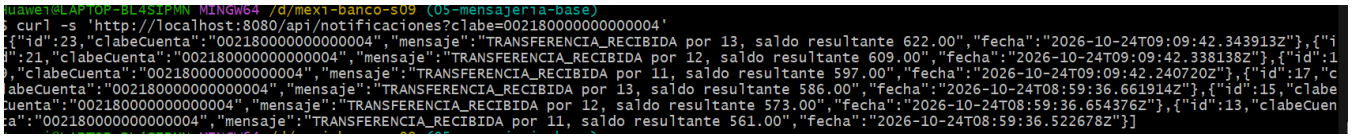


Figura. La bandeja de Beto con las tres notificaciones recibidas.

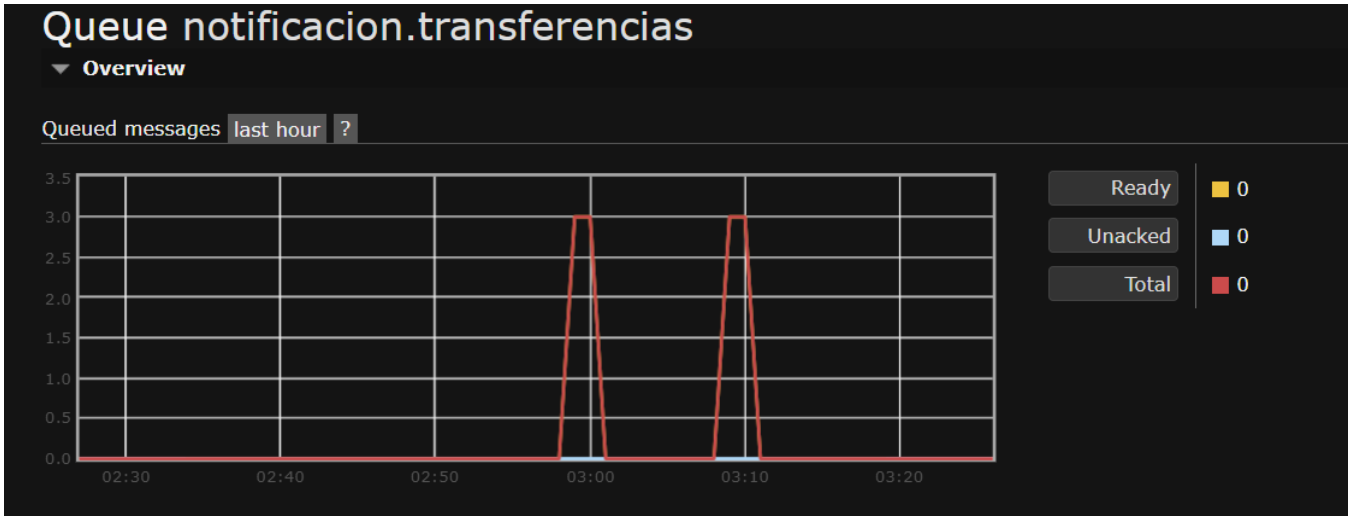


Figura . Los tres mensajes en Ready. Al encender notificación, la cola bajó de 3 a 0.

Se provocó un fallo al abonar a Beto durante una transferencia de $150. Se esperaba una respuesta HTTP 502 con estado COMPENSADA, la devolución del dinero a Ana y una notificación DEVOLUCION_TRANSFERENCIA por $150.

```
Huawei@LAPTOP-BL4SIPMN MINGW64 /d/mexi-banco-s09 (05-mensajeria-base)
$ curl -i -X POST http://localhost:8080/api/transferencias \
  -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"002180000000000003","claveDestino":"002180000000000004","monto":150}'
HTTP/1.1 502 Bad Gateway
Transfer-Encoding: chunked
Content-Type: application/problem+json
Date: Sat, 24 Oct 2026 09:15:01 GMT

{"type":"about:blank","title":"Transferencia no realizada","status":502,"detail":"No se pudo abonar a la CLABE 002180000000000004,
 asi que se devolvieron 150 a la CLABE 002180000000000003: su saldo quedo como estaba.","instance":"/transferencias","transferenci
a":26,"estado":"COMPENSADA","motivo":"El abono a la CLABE 002180000000000004 fallo: Falla provocada con FALLAR_ABONO_A_CLABE=00218
0000000000004"}
```

Figura. Respuesta de la transferencia revertida

```
Huawei@LAPTOP-BL4SIPMN MINGW64 /d/mexi-banco-s09 (05-mensajeria-base)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000003'
[{"id":24,"clabeCuenta":"002180000000000003","mensaje":"DEVOLUCION_TRANSFERENCIA por 150, saldo resultante 878.00","fecha":"2026-1
0-24T09:15:01.796157Z"},{"id":22,"clabeCuenta":"002180000000000003","mensaje":"TRANSFERENCIA_ENVIADA por 13, saldo resultante 878.
00","fecha":"2026-10-24T09:09:42.343868Z"},{"id":20,"clabeCuenta":"002180000000000003","mensaje":"TRANSFERENCIA_ENVIADA por 12, sa
ldo resultante 891.00","fecha":"2026-10-24T09:09:42.338051Z"},{"id":18,"clabeCuenta":"002180000000000003","mensaje":"TRANSFERENCIA
_ENVIADA por 11, saldo resultante 903.00","fecha":"2026-10-24T09:09:42.240414Z"},{"id":16,"clabeCuenta":"002180000000000003","mens
aje":"TRANSFERENCIA_ENVIADA por 13, saldo resultante 914.00","fecha":"2026-10-24T08:59:36.661799Z"},{"id":14,"clabeCuenta":"002180
000000000003","mensaje":"TRANSFERENCIA_ENVIADA por 12, saldo resultante 927.00","fecha":"2026-10-24T08:59:36.654313Z"},{"id":12,"c
labeCuenta":"002180000000000003","mensaje":"TRANSFERENCIA_ENVIADA por 11, saldo resultante 939.00","fecha":"2026-10-24T08:59:36.52
2196Z"}]
Huawei@LAPTOP-BL4SIPMN MINGW64 /d/mexi-banco-s09 (05-mensajeria-base)
$
```

Figura. Respuesta de la transferencia revertida bandeja de Ana

El ack se envía después de guardar para evitar perder mensajes si el servicio falla. Si se confirma antes de guardar, el mensaje puede eliminarse sin generar la notificación. Si el servicio falla después de guardar, se puede entregar nuevamente el evento y provocar notificaciones duplicadas.

PARTE 4. El duplicado

Se realizó una transferencia de $10 de la cuenta de Ana a la cuenta de Beto. Para comprobar el problema de los mensajes duplicados, se configuró una pausa de 15 segundos antes de confirmar el mensaje a RabbitMQ. Durante esa pausa, se detuvo el servicio de notificaciones y después se reinició, provocando que el mismo evento pudiera procesarse nuevamente y generar avisos duplicados tanto para Ana como para Beto.

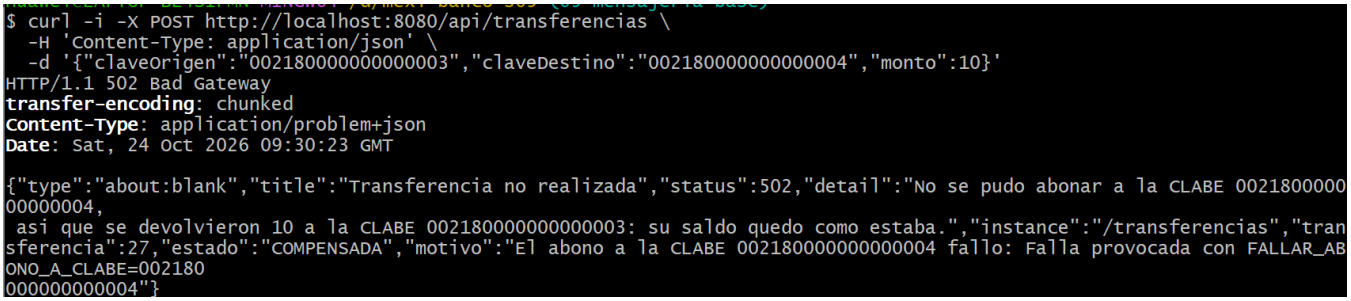


Figura. Respuesta HTTP 502 y compensación pendiente.

El resultado fue que la transferencia de $10 de Ana a Beto se revirtió. En los registros apareció el evento TransferenciaRevertida, con redelivered=false y un aviso nuevo para Ana.

$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000003'

[{"id":29,"clabeCuenta":"002180000000000003","mensaje":"TRANSFERENCIA_ENVIADA por 10, saldo resultante 848.00","fecha":"2026-10-24T09:35:10.852104Z"},{"id":28,"clabeCuenta":"002180000000000003","mensaje":"TRANSFERENCIA_ENVIADA por 10, saldo resultante 848.00","fecha":"2026-10-24T09:35:10.123542Z"}]

Al configurar la pausa antes del ack y reiniciar el servicio de notificaciones, RabbitMQ volvió a entregar el mensaje que quedó sin confirmar (estado *unacked*). Al no contar con una validación de idempotencia en la base de datos, el sistema procesó el mismo eventId dos veces}

Solucion

Se implementó idempotencia registrando cada eventoId en la base de datos antes de generar las notificaciones. Si el evento ya existe, el servicio no guarda avisos nuevos. De esta forma, aunque RabbitMQ entregue el mismo mensaje más de una vez, se evita duplicar las notificaciones.

Fragmento en EventoProcesadoRepository.java

```
@Modifying
@Query(value = """
```

```
    INSERT INTO eventos_procesados
      (evento_id, tipo, transferencia_id)
    VALUES (:eventoId, :tipo, :transferenciaId)
    ON CONFLICT (evento_id) DO NOTHING
    """, nativeQuery = true)
  int marcarSiEsNuevo(
      @Param("eventoId") UUID eventoId,
      @Param("tipo") String tipo,
      @Param("transferenciaId") Long transferenciaId);
```

Fragmento en NotificacionService.java

Al inicio de avisarDeTransferencia, antes de generar los avisos:

```
if (eventoProcesadoRepository.marcarSiEsNuevo(
      evento.eventoId(),
      evento.tipo(),
      evento.transferenciaId()) == 0) {

    log.info("[consumidor] Evento {} ya procesado; 0 aviso(s) nuevo(s)",
        evento.eventoId());

    return List.of();
  }
```

En la prueba se realizó una sola transferencia de $10 de Ana a Beto, por lo que sus saldos pasaron de $858.00 a $848.00 y de $642.00 a $652.00, respectivamente. En RabbitMQ se publicaron dos copias del mismo evento (eventoId), por lo que hubo dos mensajes Ready, pero no dos transferencias. Gracias a la idempotencia, el primer procesamiento generó dos avisos, uno para cada cuenta, mientras que el segundo detectó que el evento ya había sido procesado y generó cero avisos nuevos, evitando duplicados.

```
Huawei@LAPTOP-BL4SIPMN MINGW64 /d/mexi-banco-s09 (05-mensajeria-base)
$ curl -i -X POST http://localhost:8080/api/transferencias \
  -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"0021800000000000003","claveDestino":"0021800000000000004","monto":10}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Sat, 24 Oct 2026 09:45:54 GMT

{"id":28,"claveOrigen":"0021800000000000003","claveDestino":"0021800000000000004","monto":10,"fecha":"2026-10-24T09:45:54.477602940Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":868.00,"intentosDeCompensacion":0,"actualizada":"2026-10-24T09:45:54.729684219Z","eventoId":"5b583f04-4383-4a9b-b009-b3158f3a3565"}
Huawei@LAPTOP-BL4SIPMN MINGW64 /d/mexi-banco-s09 (05-mensajeria-base)
```

Figura. Prueba de $10 cuando notificaciones apagada

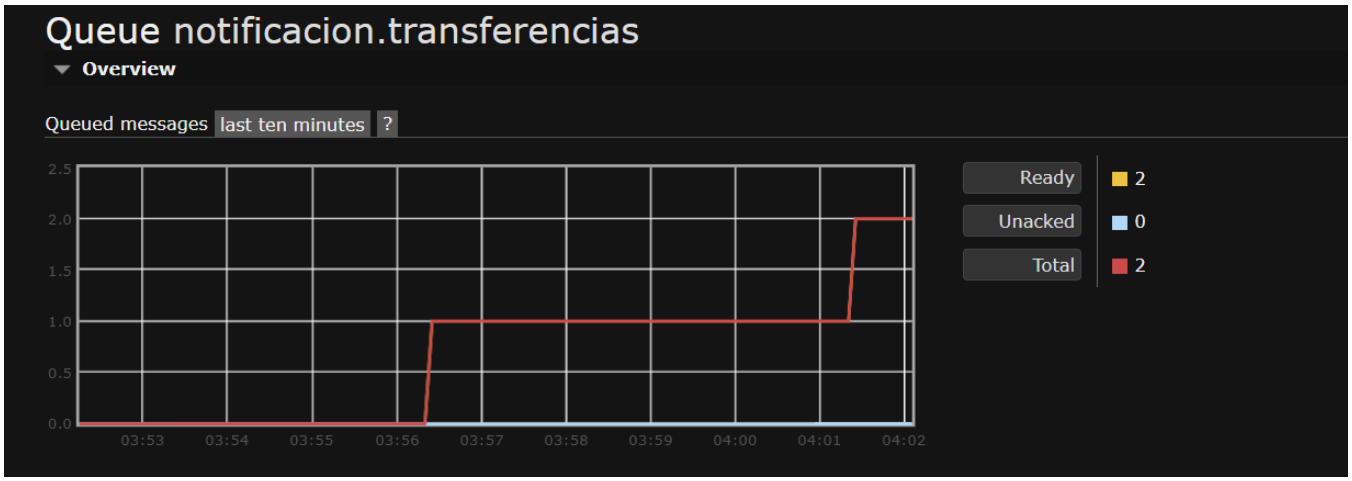


Figura. Duplicado de mensajes ready

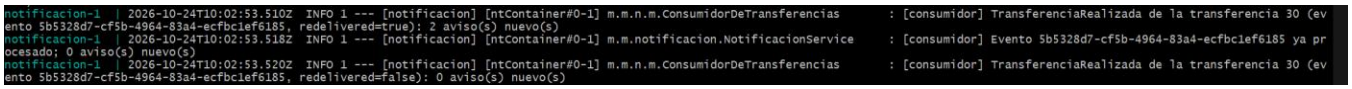


Figura. 1 solo mensaje de notificacion

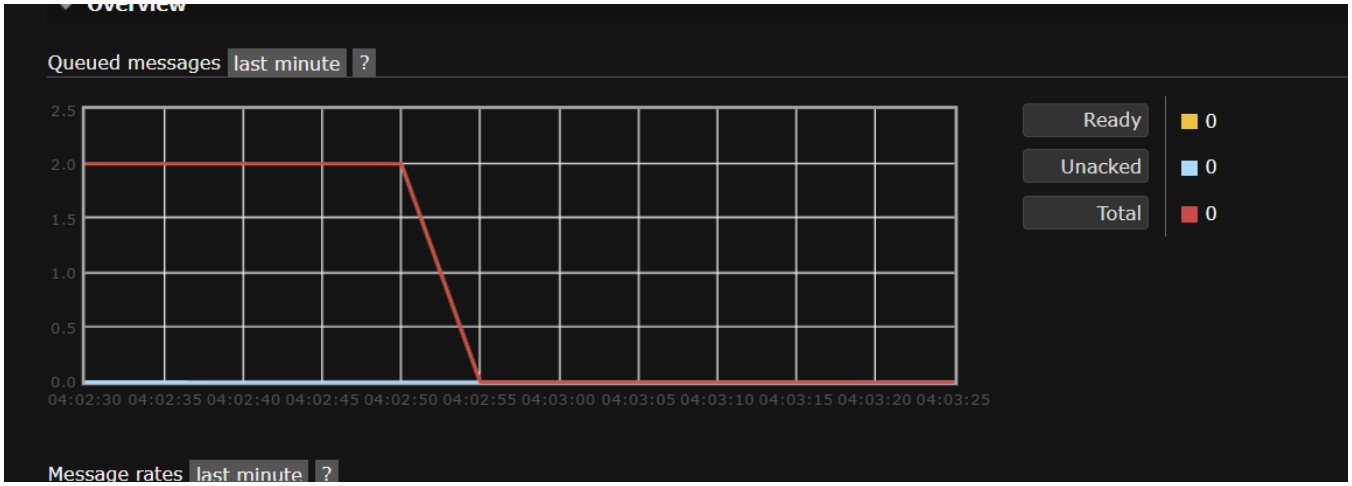


Figura. Envio de mensajes

Cuenta	Saldo antes	Movimiento	Saldo después
Ana (origen)	$858.00	−$10.00	$848.00
Beto (destino)	$642.00	+$10.00	$652.00
Total	$1,500.00	$0.00	$1,500.00

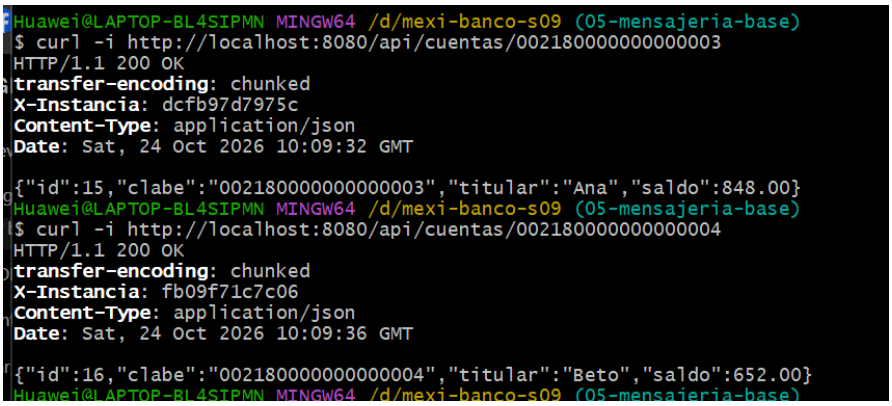


Figura. Saldos finales

PARTE 5. Mensaje Envenenado

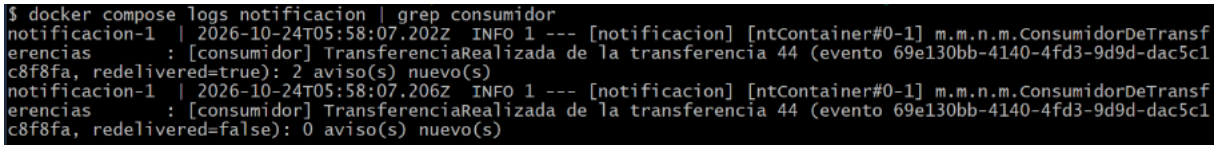


Figura. Antes

Prueba del problema de doble escritura

Para reproducir el problema, se detuvo el contenedor de RabbitMQ y posteriormente se realizó una transferencia de $1.00 mediante la API. La petición respondió con el código HTTP 201 Created y la transferencia con ID 31 quedó en estado COMPLETADA. Sin embargo, en los logs del servicio de transferencia apareció un error indicando que el evento TransferenciaRealizada no se publicó, debido a que RabbitMQ no estaba disponible.

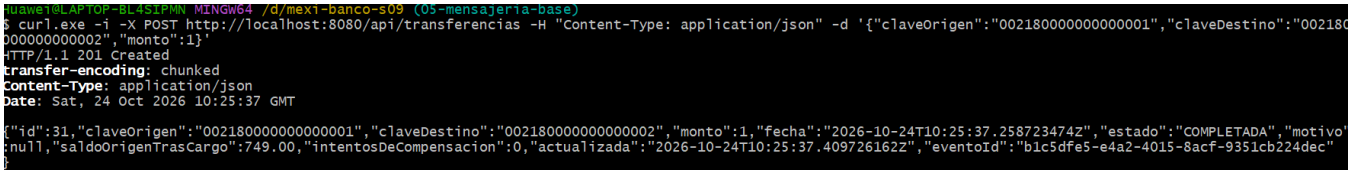


Figura . Ejecución de una transferencia con RabbitMQ detenido.

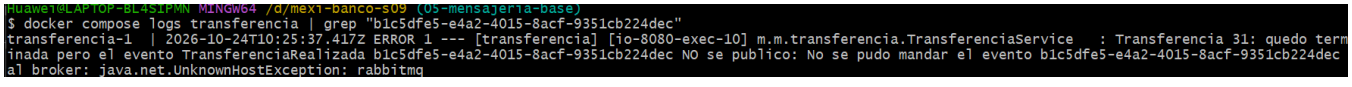


Figura . Error al publicar el evento TransferenciaRealizada.

SOLUCION:

La solución consiste en configurar una cola de mensajes muertos en RabbitMQ y modificar el consumidor para que, cuando detecte un JSON inválido, rechace el mensaje sin solicitar que vuelva a la cola. RabbitMQ utiliza la configuración de Dead Letter Exchange para redirigirlo a la DLQ.

ConfiguracionMensajeria.java

Este archivo se modificó para declarar el intercambio principal, la cola de notificaciones.

```
@Bean
public TopicExchange intercambioTransferencias() {
  return new TopicExchange("transferencias", true, false);
}

@Bean
public Queue colaTransferencias() {
  return QueueBuilder.durable("notificacion.transferencias")
      .deadLetterExchange("transferencias.dlx")
      .deadLetterRoutingKey("notificacion.transferencias")
      .build();
}

@Bean
public DirectExchange intercambioMensajesMuertos() {
  return new DirectExchange("transferencias.dlx", true, false);
}

@Bean
public Queue colaMensajesMuertos() {
  return QueueBuilder.durable("notificacion.transferencias.dlq").build();
}
@Bean
public Binding enlaceTransferencias() {
  return BindingBuilder
      .bind(colaTransferencias())
      .to(intercambioTransferencias())
      .with("transferencia.*");
}

@Bean
```

```
public Binding enlaceMensajesMuertos() {
   return BindingBuilder
         .bind(colaMensajesMuertos())
         .to(intercambioMensajesMuertos())
         .with("notificacion.transferencias");
}
```



Figura . Configuración de la cola principal y el DLX.

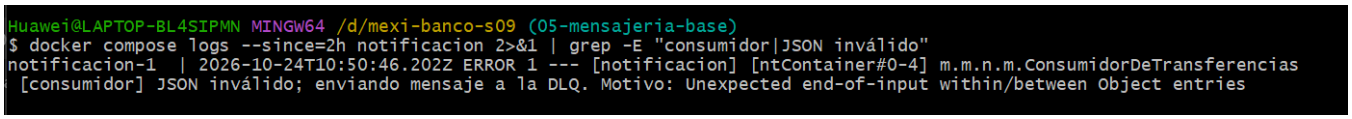


Figura . Detección del mensaje inválido en los logs.

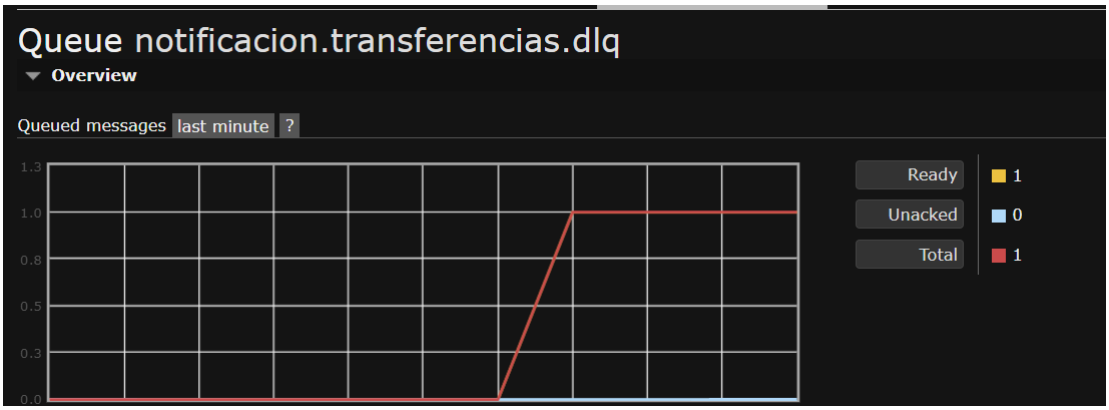


Figura . Cola de mensajes envenenados en RabbitMQ.

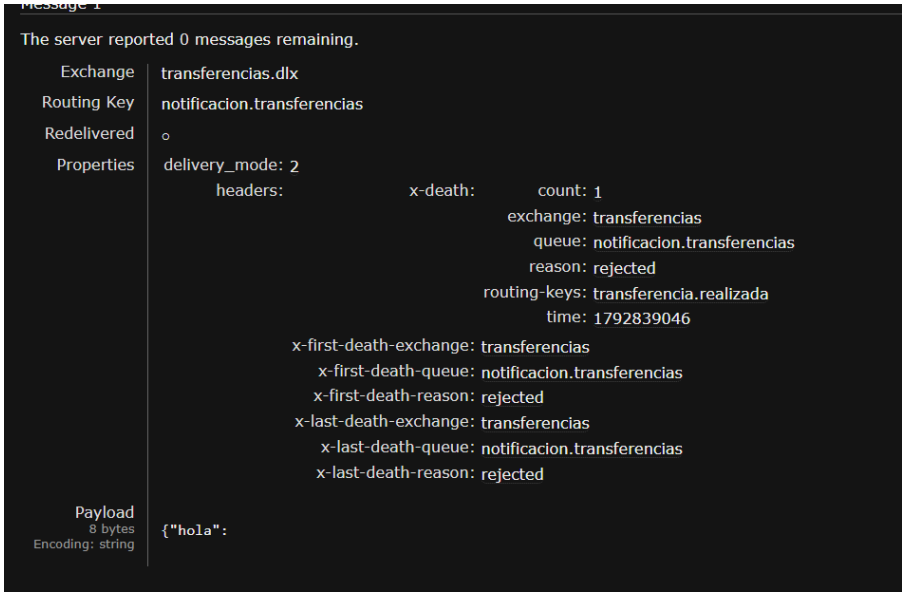


Figura . Mensaje rechazado almacenado en la DLQ.

PARTE 6. PREGUNTAS DE CIERRE

1. **Si RabbitMQ está caído cuando la saga llega a COMPLETADA, ¿qué pasa con el aviso en tu versión? ¿Por qué no lo arregla el publisher confirm?**

Cuando eso pasa, en nuestra versión el aviso puede perderse porque la transferencia ya terminó pero el mensaje no se pudo enviar. El publisher confirm no lo arregla porque solo confirma si RabbitMQ recibió el mensaje, no garantiza que se envie si el servidor está caído.

2. **¿Por qué el ack va después de guardar y no antes? ¿Qué garantía obtienes con cada orden?**

Ya que si el servicio falla antes de guardarlo, RabbitMQ puede volver a entregarlo. Si hacemos el ack antes de guardar, se puede perder el mensaje si ocurre un error.

3. **Elige y di qué sacrificaste: al quitar la llamada síncrona ganaste que el aviso no tumbe la transferencia. ¿Qué perdiste? Tres líneas: elección, porque, sacrifico.**

- Elección: quitar la llamada síncrona para que el aviso no detenga la transferencia.
- Porque: la transferencia puede terminar aunque el servicio de notificaciones falle.
- Sacrifico: que el aviso no llegue inmediatamente y que se pueda perder si no tenemos un mecanismo para reintentar el envio.